

AA Coding Interview Task - SMR Appointment Scheduler

Overview

Build a small internal scheduling application for our Service, Maintenance & Repair (SMR) team. The goal is to give us a realistic look at how you design, build, and reasoning when working alongside an AI assistant.

- Spend no more than 3 hours on the task before the interview.
- Use an AI coding assistant throughout - Claude Code, Cursor, Codex, or similar. We expect you to use AI; we want to see how you use it.
- Check your work into a public Git repository (GitHub, GitLab, or similar) and share the link before the interview.
- Be honest about what you didn't get to and why - that's part of what we're assessing.
- Please save your initial prompts and any subsequent prompting - Don't worry, it's ok for the context to go off track and for you to bring it back on track with prompting.
- Clearly outline any tooling\commands\file you might be using in your prompting.

The Application

The AA SMR team books in customer vehicles for inspections, maintenance, and repair work. Today this is tracked in spreadsheets and shared calendars, which can cause double-bookings, missed jobs, and lost notes between mechanics. You're being asked to build a minimum viable internal tool to replace this process.

Functional Requirements

Customer / booking-agent flow

- View a list of available appointment slots for the next 7 days.
- Filter slots by service type (e.g. Inspection, Service, Repair, Diagnostics)

Book an appointment by providing:

- Customer name
- Customer phone number
- Vehicle registration
- Service type
- Multi-branch / multi-location support
- Notes / description of issue
- Prevent double-booking of the same slot
- See confirmation of the booking with a unique reference number

Mechanic flow

- View a list of appointments assigned to them for today and tomorrow
- Open an appointment to see customer details, vehicle, and customer notes
- Add work notes to an appointment (free text, timestamped)
- Update appointment status: Scheduled → In Progress → Completed (or No-Show)

Admin / Shared

- A simple home page showing today's schedule across all mechanics

Out of scope - These are not required, if you start drifting into them, stop and note them as future work:

- Authentication / login (a dropdown to "act as" a user is fine for initial application)
- Email or SMS notifications
- Rescheduling and cancellation flows
- Recurring appointments
- Payments / invoicing
- Mobile-specific UI

Technical Requirements

Stack

Pick one of the following - both are equally acceptable:

- **Option A:** React frontend + C# / .NET 8 Web API backend
- **Option B:** Blazor Server (.NET 8) - single project or Blazor and REST service

Data

- Use SQL Server localhost (LocalDB or containerised) as the database.
- Use Entity Framework Core for data access (or Dapper if you prefer - your call).
- Apply schema via migrations\startup script - the app should fully set itself up on first run.
- Some seed data so the app is usable on first run (a few mechanics, a few service types, slots).

Source control

- Public Git repo with a meaningful commit history.
- We'd rather see several small, descriptive commits than one giant "initial commit". This helps us see how the work evolved.

AI Usage Expectations

We want you to use AI heavily. We're not testing whether you can write CRUD from memory. We're testing whether you can direct an AI effectively, review what it produces, and make good decisions about the result. You do of course need to understand the code.

Deliverables

In the Git repo:

- The application source code.
- docker-compose.yml and any required Dockerfiles if Docker used
- README.md covering (can it brief, 1 or 2 lines each):
 - How to run it (one command if possible)
 - Stack choice and why
 - What's done, what's not done, or what you'd do with more time
 - Any known bugs or rough edges
 - Which AI tool(s) you used.
 - What planning you did.
- The prompts you used if you wish to share - again it's ok for the AI to go off track and for you to correct this.
- Anything you deliberately wrote yourself rather than asking the AI to do.

During the Interview - We'll ask you to:

- Walk through the codebase - high level architecture, then make a few specific decisions.
- Talk through your AI usage - show us a prompt or two from your session.
- Make a small live change. We'll give you a short extension to the application. You can use AI during this part too - in fact we'd like you to.

Questions?

If anything in this brief is unclear, please do ask - that's a sensible thing to do, not a black mark.

Good luck, and have fun with it!